

Министерство образования и науки Российской Федерации
Пермский институт (филиал)
федеральное государственное бюджетное образовательное учреждение
высшего образования
«Российский экономический университет имени Г.В. Плеханова»

Сопроводительная записка

к практической работе №3

по дисциплине «Поддержка и тестирование программных модулей»

тема: «Разработка WPF-приложения для управления складом с покрытием
модульными тестами»

Выполнил:
студент группы ИПо-32
Ощепков Михаил Алексеевич

г. Пермь, 2026

Содержание

1. Введение и анализ задания.....	3
2. Выбор архитектуры и технологий.....	4
3. Проектирование модели данных	5
4. Разработка сервисного слоя	7
4.1. Интерфейс IRepository и структура сервисов	7
4.2. OrganizationService и WarehouseService	7
4.3. ProductService	8
4.4. InvoiceService и партионный учёт.....	8
5. Генерация тестовых данных	10
6. Разработка пользовательского интерфейса.....	11
7. Разработка модульных тестов.....	12
7.1. OrganizationServiceTests и WarehouseServiceTests	12
7.2. ProductServiceTests.....	12
7.3. InvoiceServiceTests	13
8. Трудности и способы их решения	15
9. Рефлексия и выводы	17

Введение и анализ задания

Тема практической работы была определена преподавателем в виде технического задания. Задание предполагало разработку WPF-приложения «Управление складом» — системы для ведения учёта организаций, складов, товаров и движения товаров посредством накладных. Моей задачей было не придумать предметную область, а вдумчиво разобраться в уже сформулированных требованиях и реализовать их в полном объёме.

Прочитав задание, я выделил несколько ключевых моментов. Во-первых, система должна быть трёхуровневой: сначала выбирается организация, затем — склад внутри организации, затем — товары на этом складе. Это не просто навигация, а структура данных: каждый уровень является родительским для следующего.

Во-вторых, задание явно указывало на партионный учёт: поступление товаров происходит партиями, каждое перемещение сопровождается накладной (входящей или исходящей), а изменение остатка происходит только после подтверждения накладной. Это потребовало реализации двух типов накладных и явного механизма их подтверждения.

В-третьих, задание содержало архитектурное требование: вся логика и модели — в отдельной библиотеке классов (.Data), каждый сервис и каждый его метод должны быть покрыты модульными тестами. Данные хранятся в коллекциях в памяти, без базы данных. Это требование сразу задало направление архитектуры и определило состав проектов в решении.

Анализ задания до начала написания кода помог мне избежать ситуации «пишу, а потом переделываю». Я чётко понимал, что именно нужно реализовать и в каком порядке стоит двигаться.

Выбор архитектуры и технологий

Технологический стек был частично определён заданием: язык C#, фреймворк WPF, рекомендуемая архитектура MVVM. Мне оставалось принять решения по тем пунктам, которые задание не оговаривало жёстко.

Решение разбить проект на три отдельных проекта прямо следует из требования задания: WarehouseManagement_Oshchepkov.Data (модели и сервисы), WarehouseManagement_Oshchepkov.WPF(интерфейс), WarehouseManagement_Oshchepkov.Tests (тесты). Такое разделение позволяет тестовому проекту ссылаться только на библиотеку данных, не затрагивая WPF-зависимости. Это принципиально важно: тесты должны запускаться без интерфейса.

Платформа .NET 8.0 выбрана как актуальная версия с поддержкой WPF. Для модульного тестирования я выбрал NUnit с атрибутами [TestFixture] и [Test] — стандартный и широко распространённый фреймворк для C#, в котором каждый тестовый класс помечается [TestFixture], метод инициализации — [SetUp], а каждый тест — [Test].

Паттерн MVVM я воспринял как практический инструмент, а не формальность. Разделение на Model, View и ViewModel даёт конкретный выигрыш: сервисный слой тестируется изолированно, ViewModel не знает о деталях отображения, View не содержит бизнес-логики. Для реализации команд в WPF я использовал RelayCommand — собственную лёгкую реализацию ICommand, которая принимает Action и предикат CanExecute.

Проектирование модели данных

Прежде чем писать сервисы, я спроектировал модели. Задание фактически диктовало их структуру — мне нужно было формализовать её в классах. Все модели находятся в пространстве имён `WarehouseManagement_Oshchepkov.Data.Models`.

Модель `Organization` содержит `Id` типа `long` и `Name`. Простая сущность — корень иерархии. Модель `Warehouse` помимо `Id` и `Name` содержит `Address` и `OrganizationId` — внешний ключ на организацию. Это явная связь, которая позволяет фильтровать склады по организации.

Модель `Product` оказалась наиболее богатой по составу полей, поскольку задание требовало конкретный набор колонок в таблице: `Article` (артикул, используется как первичный ключ), `Name`, `Unit` (единица измерения), `Price`, `DiscountPercent`, `StockQuantity`, `PhotoPath`, `Description`. Связанные сущности `Category`, `Manufacturer` и `Supplier` хранятся отдельными классами и привязываются к товару через `CategoryId`, `ManufacturerId`, `SupplierId`. Навигационные свойства `Category`, `Manufacturer` и `Supplier` позволяют обращаться к связанным объектам напрямую без дополнительных запросов.

Отдельного внимания потребовало проектирование модели накладной. Задание описывало два типа движений: поступление и списание. Я реализовал перечисление `InvoiceType` с двумя значениями — `Incoming` и `Outgoing`. Класс `Invoice` содержит `Id`, `WarehouseId`, `Date`, `Type`, флаг `IsConfirmed` и коллекцию строк `Items` типа `ObservableCollection<InvoiceItem>`. Использование `ObservableCollection` позволяет автоматически обновлять интерфейс при добавлении или удалении строк накладной — это стандартный подход в WPF.

Класс `InvoiceItem` хранит ссылку на артикул товара (`ProductArticle`), его название (`ProductName`), количество (`Quantity`) и цену на момент добавления

в накладную (Price). Фиксация цены в строке накладной — важное архитектурное решение: если цена товара изменится позже, история накладных останется корректной.

Флаг IsConfirmed в модели Invoice — ключевое проектное решение. Накладная существует в двух состояниях: не подтверждённая (черновик) и подтверждённая. Только подтверждение вызывает изменение остатков. Это прямо соответствует требованию задания: «после заполнения накладной и подтверждения перемещения товаров количество этих товаров на складе должно измениться».

Разработка сервисного слоя

Интерфейс IRepository и структура сервисов

Задание требовало создать сервисы для CRUD-операций. Чтобы унифицировать интерфейс, я ввёл обобщённый интерфейс `IRepository<T>` с методами `GetAll()`, `GetById(object id)`, `Add(T entity)`, `Update(T entity)` и `Delete(object id)`. Это стандартный паттерн репозитория, который делает сервисы предсказуемыми и облегчает написание тестов — зная интерфейс, понимаешь, что именно тестировать.

`OrganizationService` и `WarehouseService` реализуют `IRepository<T>` напрямую. `ProductService` не реализует `IRepository`, поскольку его идентификатор — строковый артикул, а не числовой `Id`, что потребовало несколько иных сигнатур методов. `InvoiceService` имеет собственный API, специфичный для логики накладных.

OrganizationService и WarehouseService

`OrganizationService` при создании сразу наполняет внутренний список через `DataGenerator.GenerateOrganizations()`. Все методы работают с приватным `List<Organization>`. `GetById()` принимает `object` и приводит его к `long` — это сделано для соответствия интерфейсу `IRepository`, где тип идентификатора не фиксирован. `Update()` находит элемент по индексу в списке и заменяет его целиком — такой подход проще, чем обновление полей по одному.

`WarehouseService` принимает `OrganizationService` в конструкторе — это внедрение зависимости. При инициализации склады генерируются через `DataGenerator.GenerateWarehouses()`, которому передаётся список организаций. Дополнительный метод `GetByOrganizationId()` позволяет получить только склады конкретной организации — именно он используется в интерфейсе при переходе от организации к её складам.

ProductService

ProductService при инициализации создаёт через DataGenerator четыре связанных справочника: категории, производителей, поставщиков и сами товары. Каждый товар при генерации получает ссылки на объекты Category, Manufacturer и Supplier — это позволяет отображать их названия в таблице без дополнительных запросов.

Особого внимания заслуживает метод UpdateStock(string article, int quantityChange). Он принимает изменение остатка со знаком: положительное значение увеличивает остаток (приходная накладная), отрицательное — уменьшает (расходная). Этот метод вызывается из InvoiceService при подтверждении накладной. Такое разделение ответственности — управление остатком находится в ProductService, а логика накладной — в InvoiceService — соответствует принципу единственной ответственности.

Метод GetProductsByWarehouse() в текущей реализации возвращает все товары с ненулевым остатком. Это рабочее решение для учебного проекта: поскольку данные хранятся в единой коллекции в памяти, а не в реляционной базе, полная фильтрация по складу потребовала бы дополнительной таблицы связей склад-товар, что выходит за рамки задания.

InvoiceService и партионный учёт

InvoiceService — наиболее сложный сервис в проекте. Он принимает ProductService в конструкторе и хранит список подтверждённых накладных в приватном List<Invoice>.

Метод CreateInvoice() создаёт новый объект Invoice с заданным типом и складом, присваивает ему уникальный Id через счётчик _nextId и возвращает его. Накладная пока не добавлена в список — она находится в состоянии черновика. Это позволяет заполнять её строки перед подтверждением.

Метод `AddItemToInvoice()` ищет товар по артикулу через `ProductService` и добавляет строку в накладную. Если товар с таким артикулом уже есть в накладной, количество суммируется — это предотвращает дублирование строк при повторном добавлении одного товара.

Метод `ConfirmInvoice()` реализует ключевую бизнес-логику. Сначала он проверяет, не подтверждена ли накладная уже — если да, возвращает `false`. Для расходной накладной выполняется предварительная проверка: по каждой строке сверяется, достаточно ли товара на складе. Если хотя бы по одной строке остатка не хватает — метод возвращает `false` без изменения остатков. Только после успешной проверки всех строк происходит обновление остатков: приходные строки увеличивают `StockQuantity`, расходные — уменьшают. Накладная помечается как подтверждённая и добавляется в список. Такая атомарная логика — «всё или ничего» — исключает ситуацию частичного проведения накладной.

Генерация тестовых данных

Задание явно требовало создать методы, заполняющие коллекции тестовыми данными. Я реализовал статический класс `DataGenerator` с пятью методами: `GenerateOrganizations()`, `GenerateCategories()`, `GenerateManufacturers()`, `GenerateSuppliers()` и `GenerateProducts()`. Метод `GenerateWarehouses()` принимает список организаций и генерирует склады с привязкой к ним.

При генерации организаций создаётся пять записей с реалистичными названиями юридических лиц. Для производителей добавлены известные бренды: `Lenovo`, `Logitech`, `Samsung`, `HP`, `Dell`, `Apple`, `Asus`, `Acer`. Это делает тестовые данные наглядными и похожими на реальный склад электроники.

Метод `GenerateProducts()` использует вспомогательные функции `GetCategoryId()`, `GetManufacturerId()`, `GetSupplierId()`, которые ищут нужный справочный объект по имени. Это удобнее, чем жёстко прописывать числовые `Id`. При генерации каждый товар сразу получает заполненные навигационные свойства — объекты `Category`, `Manufacturer` и `Supplier`, а не только их идентификаторы. Артикул формируется как шестизначное число с ведущими нулями через формат `D6`.

Класс `DataGenerator` используется исключительно в конструкторах сервисов, что обеспечивает изолированность: каждый раз при создании нового сервиса (в том числе в тестах) данные генерируются заново, независимо от предыдущих тестов.

Разработка пользовательского интерфейса

Задание содержало конкретные требования к интерфейсу: унифицированная тема через ResourceDictionary на уровне Application, отображение текущего контекста, валидация ввода. Интерфейс разбит на отдельные окна: OrganizationsWindow (список организаций), MainWorkWindow (главное рабочее окно со складами и товарами), а также диалоговые окна для создания и редактирования каждой сущности.

Диалог OrganizationDialog запрашивает название компании. WarehouseDialog — название и адрес склада. ProductDialog содержит все поля товара: название, артикул, единицу измерения, цену, скидку, категорию, производителя, поставщика, путь к фотографии. InvoiceDialog позволяет выбрать тип накладной (входящая/исходящая), добавить строки с товарами и подтвердить накладную.

Диалог ConfirmDeleteDialog реализует требование задания о подтверждении удаления организации через ввод её названия. Пользователь должен вручную написать название — только при совпадении кнопка подтверждения становится активной. Это защита от случайного удаления.

Для команд в WPF я использовал RelayCommand — собственную реализацию ICommand. Это позволяет привязывать кнопки к методам ViewModel через {Binding Command} без code-behind. Таблица товаров реализована через DataGrid с колонками согласно требованиям задания: Фото, Наименование, Категория, Производитель, Поставщик, Цена, Остаток, Скидка.

Разработка модульных тестов

Требование задания было конкретным: каждый сервис и каждый метод сервиса должны быть покрыты модульными тестами. Это не пожелание, а обязательное условие. Все тесты используют NUnit, структурированы по принципу AAA (Arrange–Act–Assert) и изолированы друг от друга: в методе [SetUp] каждого тестового класса создаются новые экземпляры сервисов.

OrganizationServiceTests и WarehouseServiceTests

OrganizationServiceTests содержит пять тестов: GetAll_ShouldReturnOrganizations проверяет, что после инициализации список не пустой; Add_ShouldIncreaseCount проверяет, что после добавления количество увеличивается на единицу; GetById_ShouldReturnCorrectOrganization проверяет поиск по Id; Update_ShouldChangeOrganizationName проверяет изменение имени; Delete_ShouldRemoveOrganization проверяет удаление и последующее отсутствие элемента при поиске по Id.

WarehouseServiceTests аналогичен по структуре, но содержит дополнительный тест GetByOrganizationId_ShouldReturnCorrectWarehouses — он берёт первую организацию и проверяет, что все возвращённые склады принадлежат именно ей. Это важный тест, проверяющий корректность фильтрации, которая лежит в основе навигации Организация → Склады.

ProductServiceTests

ProductServiceTests охватывает девять тестов. Помимо стандартных CRUD-проверок (GetAllProducts, Add, GetByArticle, Update, Delete) здесь есть тест UpdateStock_ShouldChangeQuantity — он изменяет остаток на 10 единиц и проверяет, что StockQuantity действительно увеличился на это значение. Также проверяются вспомогательные методы справочников: GetAllCategories_ShouldReturnCategories,

GetAllManufacturers_ShouldReturnManufacturers,
GetAllSuppliers_ShouldReturnSuppliers.

InvoiceServiceTests

InvoiceServiceTests — наиболее содержательный тестовый класс. Он содержит семь тестов, проверяющих все ключевые сценарии работы с накладными.

CreateInvoice_ShouldReturnNewInvoice проверяет, что созданная накладная не является null, имеет правильный тип, Id склада и флаг IsConfirmed = false. AddItemToInvoice_ShouldAddItem проверяет, что после добавления строки в накладную список Items содержит ровно один элемент с правильным артикулом и количеством. RemoveItemFromInvoice_ShouldRemoveItem добавляет строку и сразу удаляет её, проверяя что список пуст.

ConfirmIncomingInvoice_ShouldIncreaseStock — ключевой тест для приходной накладной. Он запоминает начальный остаток, создаёт приходную накладную, добавляет 10 единиц товара, подтверждает и проверяет три вещи: метод вернул true, флаг IsConfirmed стал true, остаток увеличился ровно на 10.

ConfirmOutgoingInvoice_ShouldDecreaseStock аналогичен, но для расходной накладной: проверяет, что остаток уменьшился на списанное количество. Тест ConfirmOutgoingInvoice_WithInsufficientStock_ShouldFail проверяет защиту от списания больше, чем есть: устанавливает количество в строке накладной заведомо больше остатка (initialStock + 100), подтверждает — и проверяет, что метод вернул false, IsConfirmed остался false, а остаток не изменился. Это тест граничного условия, который проверяет, что механизм защиты реально работает.

GetAllInvoices_ShouldReturnInvoices создаёт две накладные (приходную и расходную), подтверждает обе и проверяет, что список содержит не менее двух записей.

Трудности и способы их решения

Первая трудность — проектирование `InvoiceService`. Главный вопрос: как `ConfirmInvoice()` должен работать с расходной накладной, если остатка не хватает? Я рассматривал два варианта: выбросить исключение или вернуть `false`. Выбрал возврат `false` — это проще тестировать (не нужен блок `try/catch` в тесте) и удобнее обрабатывать в интерфейсе. Тест `ConfirmOutgoingInvoice_WithInsufficientStock_ShouldFail` подтверждает, что этот сценарий обработан корректно.

Вторая трудность — привязки данных в WPF. Несколько раз свойство в `ViewModel` менялось, но интерфейс не обновлялся. Причина — отсутствие реализации `INotifyPropertyChanged`. Решение: базовый класс `ViewModelBase` с методом `OnPropertyChanged()`, от которого наследуют все `ViewModel`. Диагностика — окно `Output` в `Visual Studio`, куда WPF записывает ошибки привязок при отладке.

Третья трудность — обновление родительского окна после закрытия дочернего. Когда диалог редактирования закрывается, главное окно должно обновить список. Я решил это через обработку события `Closed` дочернего окна в `code-behind` родительского: при закрытии диалога `ViewModel` перезапрашивает данные из сервиса и обновляет коллекцию. Это не идеальное MVVM-решение, но понятное и рабочее.

Четвёртая трудность — тест, который проходил по неверной причине. При первой версии теста `ConfirmOutgoingInvoice_WithInsufficientStock_ShouldFail` я проверял только, что `result == false`, но не проверял, что остаток не изменился. Когда я добавил проверку остатка, тест всё равно прошёл — потому что метод `ConfirmInvoice()` корректно не менял остаток при возврате `false`. Это хорошо: тест подтвердил, что логика реализована верно. Но урок важен: тест должен

проверять все три аспекта — возвращаемое значение, флаг IsConfirmed и состояние данных.

Вывод

Работа с техническим заданием от преподавателя дала специфический опыт: нужно было не придумывать задачу, а читать и интерпретировать уже сформулированные требования. Это навык, который в реальной разработке применяется постоянно — техническое задание всегда пишет кто-то другой.

Первое, что я понял: вдумчивый анализ задания до написания кода — это не потеря времени, а экономия его. Понимание того, что накладная должна иметь два состояния и два типа, что остаток меняется только после подтверждения, что удаление организации подтверждается вводом названия — всё это нужно было извлечь из текста задания до написания первой строки кода.

Второе: требование «каждый метод сервиса покрыть тестом» изменило подход к проектированию. Зная, что метод будет тестироваться, проектируешь его проще и предсказуемее: один метод — одна ответственность, чёткие входные условия, явный возврат результата вместо исключений там, где это уместно.

Третье: интерфейс `IRepository<T>` дал неожиданный практический выигрыш. Одинаковая структура методов у `OrganizationService` и `WarehouseService` означает, что тестовые классы для них устроены одинаково — написал один, написал шаблон для второго.

Четвёртое: тест
`InvoiceServiceTests.ConfirmOutgoingInvoice_WithInsufficientStock_ShouldFail` оказался самым ценным. Именно граничные тесты — те, которые проверяют что система делает при некорректных данных — дают уверенность в надёжности кода. Позитивные тесты («всё работает при правильных данных») проще написать и проще пройти. Негативные тесты — настоящая проверка.